YZT101 • HAFTA 1

Programlamaya ve Algoritmik Düşünmeye Giriş

Problem → Algoritma → Akış Diyagramı → İlk Python Programı

Öğr. Gör. Mert Hızmalı
Yapay Zeka Operatörlüğü

4 ders saati • 2 teori + 2 uygulama

4 DERS SAATLIK AKIŞ

Bu hafta nereden nereye gideceğiz?

1 Programlama Mantığı

Program, dil, algoritma, problem çözme

2 Problemi Parçalama

Girdi • İşlem • Çıktı • yalancı kod

3 Akış Diyagramları

Semboller, akış ve karar noktaları

4 İlk Python Programı

Ortamı tanıma, çalıştırma, hata görme

Bugünün ana fikri

Kod yazmak son adımdır. Önce problemi anlamak, sonra çözüm adımlarını tasarlamak gerekir.

HAFTANIN HEDEFLERI

Bu hafta sonunda ne yapabiliyor olacaksınız?

01 Program, programlama dili ve algoritma kavramlarını birbirinden ayırmak

02 Bir problemi girdi–işlem–çıktı biçiminde çözümlemek

03 Basit bir çözümü yalancı kodla adım adım ifade etmek

04 Temel akış diyagramı sembollerini doğru yerde kullanmak

05 **Bir Python dosyasını oluşturup ilk programı çalıştırmak**

Bugün “çok kod” değil, doğru düşünme alışkanlığı kazanıyoruz.

BAĞLANTIYI KURALIM

Programlama neden Yapay Zeka Operatörlüğünün temelidir?



Programlama, yapay zekâ aracını “kullanmak” ile onu bir iş akışına “entegre etmek” arasındaki köprüdür.

1

1. Programlama Mantığı

Program, programlama dili ve algoritma kavramlarını birbirinden ayırıyoruz.

TEMEL KAVRAM

Program nedir?

Program **Bilgisayara belirli bir işi yaptırmak için hazırlanmış, çalıştırılabilir komutlar bütünüdür.**

Bir tarayıcı

Web sayfalarını getirir, işler ve kullanıcıya gösterir.

Bir mobil uygulama

Kullanıcı etkileşimlerini alır ve kurallara göre sonuç üretir.

Basit bir Python dosyası

Bir veya birkaç komutu sırayla çalıştırabilir.

Programın büyüklüğü değişebilir; temel fikir değişmez: bilgisayara açık talimat vermek.

İNSAN NIYETI → BILGISAYAR KOMUTU

Programlama dili neden var?



Programlama dili, insanın çözümünü bilgisayara kesin ve kurallı biçimde anlatmasını sağlar.

Algoritma nedir?

Algoritma = Bir problemi çözmek veya bir işi tamamlamak için izlenen sonlu, sıralı ve açık adımlar bütünü.



Algoritma bir programlama diline bağlı olmak zorunda değildir.

BELIRSIZLIĞI YAKALA

Etkinlik: “Sandviç yap” yeterli bir algoritma mı?

Talimat

1. Ekmeği al
2. Malzemeleri koy
3. Sandviçi hazırla

Bu adımlarda bilgisayarın sorabileceği belirsiz soruları bulun.

“Ekmeği al”

Kaç dilim? Hangi ekmek? Nereden?

“Malzemeleri koy”

Hangi malzeme? Ne kadar? Hangi sırada?

“Hazırla”

Kesilecek mi? Isıtılacak mı? Ne zaman tamam?

Ana ders

İnsan bağlamı tamamlar; bilgisayar açık talimat ister.

Sorun “çok kısa olması” değil; ölçülebilir ve uygulanabilir olmaması.

ÜÇ KAVRAMI AYIRALIM

Program, kod ve algoritma aynı şey değildir

ALGORİTMA

Çözümün mantığı ve adımlarıdır.

Dilden bağımsız olabilir.

“Ne yapacağız?” sorusuna cevap verir.

KOD

Algoritmanın belirli bir programlama dilindeki ifadesidir.

Python sözdizimine uyar.

“Bunu nasıl yazacağız?” sorusuna cevap verir.

PROGRAM

Çalıştırılabilir hale gelmiş kodlar ve ilgili bileşenlerin bütünüdür.

Bilgisayarda gerçek işi yapar.



2

2. Problemi Parçalamak

Girdi–İşlem–Çıktı modeli ve yalancı kodla çözüm tasarımı.

BIR PROGRAMI ÜÇ SORUYLA ÇÖZÜMLE

Girdi – İşlem – Çıktı

1

GİRDİ

Programa ne geliyor?

Kullanıcı verisi • dosya • sensör • sabit bilgi

2

İŞLEM

Bu veriyle ne yapıyoruz?

Hesaplama • karşılaştırma • dönüştürme • düzenleme

3

ÇIKTI

Program ne üretiyor?

Ekran sonucu • dosya • bildirim • yeni veri

Bir problem cümlesini okurken önce bu üç kutuyu doldurabiliyorsanız, çözümün yarısını kurmuşsunuz demektir.

GIRDI–İŞLEM–ÇIKTI ÖRNEKLERI

Aynı model, farklı problemler

Problem	Girdi	İşlem	Çıktı
İki sınavın ortalaması	İki sınav notu	Topla ve ikiye böl	Ortalama
Kargo ücreti hesabı	Ağırlık, mesafe	Kurallara göre ücret hesapla	Kargo ücreti
Metin uzunluğu	Bir metin	Karakterleri say	Karakter sayısı
Yapay zekâ sınıflandırması	Müşteri yorumu	Modelle sınıflandır	Etiket / kategori

Dikkat: “İşlem” her zaman matematiksel işlem olmak zorunda değildir.

ÖRNEK: ÖĞRENCI NOT ORTALAMASI

Problem metnini parçalayalım

“Bir öğrencinin birinci ve ikinci sınav notlarını al, iki notun ortalamasını hesapla ve sonucu kullanıcıya göster.”

GİRDİ

- 1. sınav notu
- 2. sınav notu

→

İŞLEM

- İki notu topla
- Sonucu 2’ye böl

→

ÇIKTI

- Hesaplanan ortalama

Kod yazmadan önce çözümün iskeleti hazır.

3 KISA SENARYO

Etkinlik 1: Girdi–İşlem–Çıktıyı siz bulun

A Bir dikdörtgenin kısa ve uzun kenarını al, çevresini hesapla ve göster.

B Bir kelimeyi al, kelimenin kaç karakter olduğunu bul ve sonucu göster.

C Bir sıcaklık değerini al, 0°C'nin altında olup olmadığını kontrol et ve uygun mesaj üret.

Her senaryo için: Girdi? → İşlem? → Çıktı?

CEVAPLAR

Etkinlik 1: Birlikte kontrol

	Girdi	İşlem	Çıktı
A	Kısa kenar, uzun kenar	2 × (kısa + uzun)	Çevre
B	Kelime	Karakterleri say	Karakter sayısı
C	Sıcaklık	0 ile karşılaştır	Uygun mesaj

C senaryosunda “karar” fikri var; Python sözdizimini daha sonra öğreneceğiz.

KONTROL LISTESI

İyi bir algoritmanın özellikleri

Açık

Her adım ne yapılacağını net söylemeli.

Sıralı

Adımlar doğru yürütme sırasına sahip olmalı.

Sonlu

Algoritma bir noktada tamamlanmalı.

Doğru

Verilen problem için beklenen sonucu üretmeli.

Uygulanabilir

Her adım gerçekten gerçekleştirilebilir olmalı.

“Çalışıyor” + “doğru çalışıyor” aynı şey değildir.

UYGULANABILIR YÖNTEM

Bir problemi algoritmaya çevirmenin 5 adımı

1	**Problemi kendi cümlenle açıkla**	Aslında ne isteniyor?
2	**Girdileri belirle**	Hangi bilgiler gerekli?
3	**Çıktıyı belirle**	Sonunda ne üretilecek?
4	**İşlemleri sırala**	Girdiden çıktıya nasıl gideriz?
5	**Örnekle test et**	Adımlar gerçekten işe yarıyor mu?

Yalancı kod (pseudocode) nedir?

NE DEĞİLDİR?

- Python değildir
- Katı sözdizimi yoktur
- Bilgisayar tarafından çalıştırılmaz
- Noktalama ezberi değildir

NEDİR?

- Çözüm adımlarını açıklar
- Günlük dile yakındır
- Sıralı ve nettir
- Koda geçişi kolaylaştırır

NEDEN KULLANIRIZ?

- Koddan önce düşünmek
- Algoritmayı tartışmak
- Dil bağımsız tasarlamak
- Mantık hatasını erken görmek

Yalancı kodun “tek doğru yazımı” yoktur; önemli olan açık ve tutarlı olmasıdır.

Yalancı kod örneği

YALANCI KOD

```
BAŞLA
  1. sınav notunu OKU
  2. sınav notunu OKU
  iki notu TOPLA
  toplamı 2'ye BÖL
  ortalamayı YAZ
BİTİR
```

1 • Girdi

İki sınav notunu alıyoruz.

2 • İşlem

Toplama ve bölme adımlarını uyguluyoruz.

3 • Çıktı

Ortalamayı kullanıcıya gösteriyoruz.

Henüz Python yazmadık; ama çözümümüz neredeyse hazır.

Etkinlik 2: Yalancı kodu siz yazın

Problem: Kısa ve uzun kenar bilgilerini al. Dikdörtgenin çevresini hesapla. Sonucu ekrana yaz.

GİRDİ	İŞLEM	ÇIKTI
?	?	?

Önce G–İ–Ç tablosunu kurun, sonra 5–7 satırlık yalancı kod yazın.

Etkinlik 2: Örnek çözüm

YALANCI KOD

```
BAŞLA
  kısa kenarı OKU
  uzun kenarı OKU
  iki kenarı TOPLA
  toplamı 2 ile ÇARP
  çevreyi YAZ
BİTİR
```

Alternatif çözüm de mümkündür:

ALTERNATİF İŞLEM

```
çevre = kısa×2 + uzun×2
```

Önemli fikir

Farklı algoritmalar aynı doğru sonuca ulaşabilir.

Doğruluk + açıklık + uygulanabilirlik → iyi çözüm

DAHA KODA GEÇMEDEN YAKALA

Yalancı kodda sık yapılan hatalar

Belirsiz adım

“Gerekli işlemleri yap.”
Hangi işlemler?

Eksik girdi

“Ortalama hesaplanıyor ama notlar alınmıyor.”
Veri nereden geliyor?

Yanlış sıra

“Sonuç hesaplanmadan ekrana yazılıyor.”
Önce ne olmalı?

Bitiş yok

“Tekrar var ama ne zaman duracağı belli değil.”
Ne zaman tamam?

Yalancı kodun amacı güzel görünmek değil, mantığı görünür hale getirmektir.

3

3. Akış Diyagramları

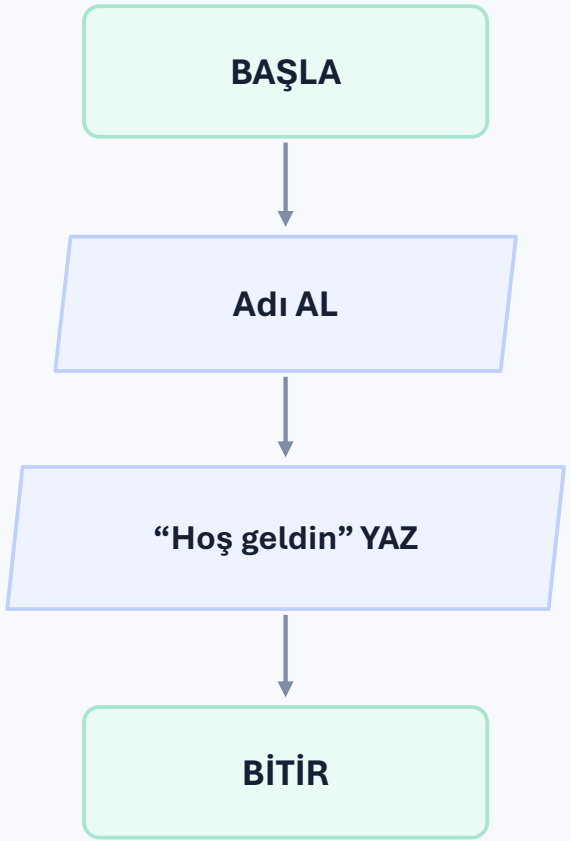
Algoritmayı görsel bir yürütme akışına dönüştürüyoruz.

ŞEKIL NE ANLATIYOR?

Akış Şemasında Kullanılan Şekiller

Dikdörtgen

Akış diyagramı nasıl okunur?



Okurken sor

1. Nereden başlıyorum?
2. Hangi veri geliyor?
3. Sonra hangi adım var?
4. Nerede bitiyor?

Bu şemada karar yok; akış tek yol üzerinden ilerliyor.

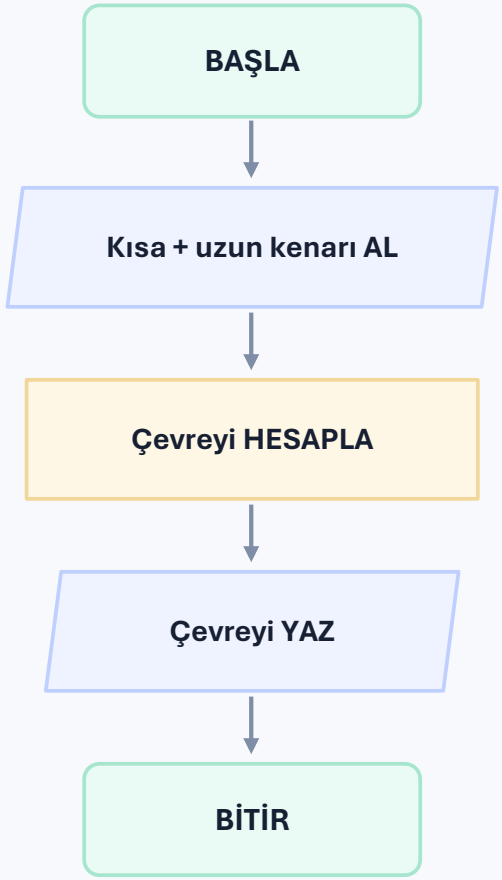
AYNI MANTIK, FARKLI GÖSTERİM

Yalancı koddan akış diyagramına

YALANCI KOD

```
BAŞLA
  kısa kenarı OKU
  uzun kenarı OKU
  çevreyi HESAPLA
  çevreyi YAZ
BİTİR
```

Aynı algoritma



Metinle düşünmek kolay olabilir; görsel akış kararları ve sıralamayı daha görünür yapar.

ÇIZDIKTEN SONRA 6 SORU

Akış diyagramı kontrol listesi

- ✓ Tek ve anlaşılır bir başlangıç var mı?
- ✓ Karar kutularının yolları etiketli mi?
- ✓ Oklar yürütme sırasını açık gösteriyor mu?
- ✓ Her olası yol mantıklı bir sonuca ulaşıyor mu?
- ✓ Girdi/çıktı ve işlem sembolleri doğru mu?
- ✓ Şemayı başka biri açıklama olmadan okuyabilir mi?

İyi diyagram “güzel” olmaktan önce anlaşılır olmalıdır.

4

4. Python ile İlk Program

Programlama ortamını tanıyor, dosya oluşturuyor ve ilk çıktıyı görüyoruz.

İLK BAKIŞ

Python nedir? Neden Python?

OKUNABİLİR

Sözdizimi görece sadedir. Kodun mantığını takip etmek başlangıçta daha kolaydır.

GENEL AMAÇLI

Otomasyon, web, veri işleme, bilimsel hesaplama ve daha birçok alanda kullanılabilir.

YZ EKOSİSTEMİ

Veri bilimi ve yapay zekâ araçlarıyla güçlü bir kütüphane ve araç ekosistemine sahiptir.

Ama unutmayın: Önce programlama mantığını öğreniyoruz, sonra Python bu mantığın dili oluyor.

BASITLEŞTIRILMIŞ YÜRÜTME MODELI

Python kodu nasıl çalışır?

1 • KAYNAK DOSYA

Programı `.py` uzantılı bir dosyaya yazarız.

Örnek: `merhaba.py`

2 • PYTHON

Python yorumlayıcısı kodu okur ve çalıştırır.

Kurallara aykırılık varsa hata verir.

3 • SONUÇ

Ekranda çıktı görebiliriz, dosya oluşabilir veya başka bir işlem gerçekleşebilir.

Bugün: kaynak dosya → Python → ekranda çıktı

Çalışma ortamında hangi parçalar var?

KOD EDİTÖRÜ

Kodu yazdığımız alan.

IDLE, Thonny, VS Code veya başka bir editör olabilir.

Önemli olan Python dosyasını düzenleyebilmek.

`.py` DOSYASI

Kodun kaydedildiği dosya.

Örnek:
`01_merhaba.py`

Dosya adı anlamlı olmalı.

ÇALIŞTIRMA / TERMİNAL

Programı Python ile çalıştırırız.

Editörün “Run” düğmesi veya terminal kullanılabilir.

Sonucu burada gözlemleriz.

Araç değişebilir; “yaz → kaydet → çalıştır → gözle” döngüsü değişmez.

BIR SATIR, GERÇEK ÇIKTI

İlk Python programımız

01_merhaba.py

```
print("Merhaba, Python!")
```

ÇIKTI

Merhaba, Python!

1 • Yaz

Kod editöründe satırı oluştur.

2 • Kaydet

Dosyayı `01_merhaba.py` olarak kaydet.

3 • Çalıştır

Programı çalıştır ve çıktıyı gözle.

SÖZDIZIMINE ILK BAKIŞ

Bu tek satırda ne görüyoruz?

PYTHON

```
print("Merhaba, Python!")
```

`print`

İşlev / fonksiyon adı

Ekrana çıktı vermek için çağırıyoruz.

`( )`

Parantez

Fonksiyona verilen bilgiyi sınırlar.

`"..."`

Tırnaklar

İçeridekinin bir metin olduğunu belirtir.

`Merhaba, Python!`

Metin değeri

Ekranda görmeyi istediğimiz içeriktir.

PROGRAMCININ TEMEL DÖNGÜSÜ

Yaz → Kaydet → Çalıştır → Gözle → Düzelt



Bir kez değil, sürekli tekrar eder.

Hata yapmak programlamanın parçasıdır

HATALI KOD

```
print("Merhaba, Python!)
```

PYTHON MESAJI

```
SyntaxError: ...
```

Tırnak açıldı ama kapanmadı.

Ne yapacağız?

1. Mesajı okuyacağız.
2. İşaret edilen satıra bakacağız.
3. Parantez/tırnak gibi yapıları kontrol edeceğiz.
4. Düzelterek yeniden çalıştıracağız.

Hata mesajı “ceza” değil, teşhis bilgisidir.

TAHMIN ET → ÇALIŞTIR → KARŞILAŞTIR

Hata avı: Hangileri çalışır?

A

```
A) print("Merhaba")
```

B

```
B) print(Merhaba)
```

C

```
C) print("Merhaba"
```

A

Tırnak ve parantez dengeli.
Çalışır.

B

Bugün için sorunlu: Merhaba metin olarak tırnak içinde değil.

C

Kapanış parantezi eksik.
Sözdizimi hatası.

Önce tahmin edin. Sonra gerçekten çalıştırın. Sonucu tahmininizle karşılaştırın.

Uygulama: 4 satırda kendini tanıt

02_kendini_tanit.py

```
print("Merhaba! Ben Mert.")
print("Yapay Zekâ Operatörlüğü dersindeyim.")
print("Bu dönem Python öğreneceğim.")
print("İlk programım çalışıyor!")
```

Görev

Dört satırı kendi bilgilerinize göre değiştirin.

Kural

Yalnızca `print` ve metin kullanın. Değişken yok.

Kontrol

Kaydet → çalıştır → dört satır çıktıyı doğrula.

Mini bütünleşik görev: “Akıllı Karşılama”

1 **Yalancı kod** — Program üç farklı karşılama mesajını sırayla gösterecek.

2 **Akış diyagramı** — Başla → üç çıktı adımı → Bitir.

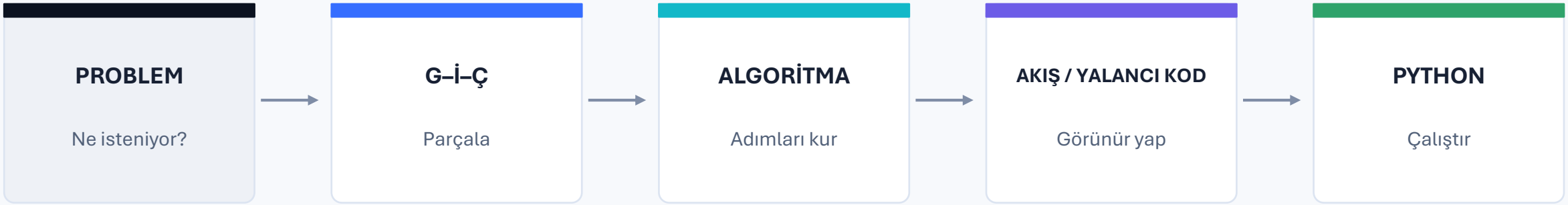
3 **Python** — Her mesaj için bir `print()` satırı yaz.

4 **Test** — Çıktı sırası beklediğin gibi mi?

Amaç: Aynı çözümü üç farklı gösterimde ifade edebilmek.

BIR PROBLEM NASIL PROGRAMA DÖNÜŞÜR?

Bugünün büyük resmi



İyi programcı önce “hangi kodu yazayım?” değil, “problemi nasıl çözmeliyim?” diye sorar.

DERSI KAPATMADAN 5 SORU

Çıkış bileti

1. Algoritma ile Python kodu arasındaki temel fark nedir?

2. “Bir ürün fiyatına KDV ekle ve sonucu göster” probleminde girdi–işlem–çıktı nedir?

3. Akış diyagramında karar için hangi sembol kullanılır?

4. `print("Merhaba")` satırında tırnakların görevi nedir?

5. Program çalışmadığında ilk yapacağınız üç kontrol ne olur?

Cevaplarınızı kısa tutun; amaç hangi kavramın oturduğunu görmek.

Gelecek hafta

Değişkenler

Bir değeri isimle saklamak

Temel veri türleri

Sayı • metin • mantıksal değer

Atama

Bir değişkene değer vermek

print() ve input()

Çıktı üretmek ve kullanıcıdan veri almak

Tür dönüşümü

Metni sayıya, sayıyı metne çevirmek

Adlandırma ve yorumlar

Okunabilir kodun ilk kuralları

Ön hazırlık: Bu haftaki `.py` dosyalarını bir kez daha çalıştırın; hata örneğini kendiniz düzeltin.

Kaynaklar ve ders materyali

Ana kaynak

Downey, A. B. (2024). Think Python: How to Think Like a Computer Scientist (3rd ed.). O’Reilly Media.

Önerilen kaynak

Matthes, E. (2023). Python Crash Course: A Hands-On, Project-Based Introduction to Programming (3rd ed.). No Starch Press.

Çevrim içi başvuru

Python Software Foundation — Python 3 Documentation / Tutorial.

Sunum içeriği YZT101 ders kataloğunun 1. hafta konu ve uygulama çerçevesiyle uyumludur.